

```

In [1]: import os
import numpy as np
import random
from PIL import Image
from sklearn.model_selection import train_test_split
import tensorflow as tf
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Input, Dense, Flatten, Reshape, BatchNormaliza
from tensorflow.keras.optimizers import Adam
from skimage.metrics import structural_similarity as ssim
import matplotlib.pyplot as plt

# Function to Load images from a directory
def load_images_from_folder(folder, target_size=(128, 128)):
    images = []
    for filename in os.listdir(folder):
        img_path = os.path.join(folder, filename)
        try:
            img = Image.open(img_path).convert('L') # Convert to grayscale
            img = img.resize(target_size) # Resize to target size
            img = np.array(img).astype('float32') / 255.0 # Normalize to [0,
            images.append(img)
        except Exception as e:
            print(f"Error loading image {filename}: {e}")
    return np.array(images)

# Encrypt image using XOR encryption
def encrypt_image(image, key):
    np.random.seed(key) # Seed for reproducibility
    noise = np.random.rand(*image.shape) # Generate random noise
    return np.mod(image + noise, 1.0) # Add noise and take modulo to stay in

# Decrypt image using reverse XOR encryption
def decrypt_image(encrypted_image, key):
    np.random.seed(key) # Seed for reproducibility
    noise = np.random.rand(*encrypted_image.shape) # Generate same noise as e
    return np.mod(encrypted_image - noise, 1.0) # Subtract noise and take mod

# Build the autoencoder model with BatchNorm & LeakyReLU
def build_autoencoder(input_shape, latent_dim=128):
    # Encoder
    input_img = Input(shape=input_shape)
    x = Flatten()(input_img)
    x = Dense(512)(x)
    x = BatchNormalization()(x)
    x = LeakyReLU()(x)
    x = Dense(latent_dim)(x)
    x = BatchNormalization()(x)
    x = LeakyReLU()(x)
    encoder = Model(input_img, x, name="encoder")

    # Decoder
    latent_input = Input(shape=(latent_dim,))
    x = Dense(512)(latent_input)

```

```
x = BatchNormalization()(x)
x = LeakyReLU()(x)
x = Dense(np.prod(input_shape), activation='sigmoid')(x)
output_img = Reshape(input_shape)(x)
decoder = Model(latent_input, output_img, name="decoder")

# Autoencoder
autoencoder_input = Input(shape=input_shape)
encoded = encoder(autoencoder_input)
decoded = decoder(encoded)
autoencoder = Model(autoencoder_input, decoded, name="autoencoder")

return autoencoder, encoder, decoder

# Metrics calculation
def calculate_metrics(original_images, reconstructed_images):
    mse_values, ssim_values = [], []
    for orig, recon in zip(original_images, reconstructed_images):
        mse = np.mean((orig - recon) ** 2)
        ssim_value = ssim(orig, recon, data_range=orig.max() - orig.min())
        mse_values.append(mse)
        ssim_values.append(ssim_value)
    return np.mean(mse_values), np.mean(ssim_values), mse_values, ssim_values

# Display random image comparisons
def display_random_comparisons(original, encrypted, decrypted, num_samples=5):
    indices = random.sample(range(len(original)), num_samples) # Randomly sel
    plt.figure(figsize=(15, 5))

    for i, idx in enumerate(indices):
        # Original
        plt.subplot(3, num_samples, i + 1)
        plt.imshow(original[idx], cmap='gray', vmin=0, vmax=1)
        plt.title("Original")
        plt.axis('off')

        # Encrypted
        plt.subplot(3, num_samples, i + 1 + num_samples)
        plt.imshow(encrypted[idx], cmap='gray', vmin=0, vmax=1)
        plt.title("Encrypted")
        plt.axis('off')

        # Decrypted
        plt.subplot(3, num_samples, i + 1 + 2 * num_samples)
        plt.imshow(decrypted[idx], cmap='gray', vmin=0, vmax=1)
        plt.title("Decrypted")
        plt.axis('off')

    plt.tight_layout()
    plt.show()

# Plot training Loss
def plot_training_loss(history):
    plt.figure(figsize=(8, 5))
    plt.plot(history.history['loss'], label='Training Loss')
    plt.plot(history.history['val_loss'], label='Validation Loss')
```

```
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.title('Autoencoder Training Loss')
plt.legend()
plt.grid()
plt.show()

# Plot MSE & SSIM distributions
def plot_metrics_distribution(mse_values, ssim_values):
    plt.figure(figsize=(12, 5))

    # MSE distribution
    plt.subplot(1, 2, 1)
    plt.hist(mse_values, bins=20, color='skyblue', edgecolor='black')
    plt.xlabel('MSE')
    plt.ylabel('Frequency')
    plt.title('MSE Distribution')

    # SSIM distribution
    plt.subplot(1, 2, 2)
    plt.hist(ssim_values, bins=20, color='salmon', edgecolor='black')
    plt.xlabel('SSIM')
    plt.ylabel('Frequency')
    plt.title('SSIM Distribution')

    plt.tight_layout()
    plt.show()

# Load dataset
dataset_folder = "C:/Users/hitha sunil/Desktop/DOTA/DOTA/DOTA/DOTA_FULL" # Re
images = load_images_from_folder(dataset_folder)

# Split dataset into training and testing
X_train, X_test = train_test_split(images, test_size=0.2, random_state=42)

# Build and compile the autoencoder
input_shape = X_train[0].shape
autoencoder, encoder, decoder = build_autoencoder(input_shape)
autoencoder.compile(optimizer=Adam(learning_rate=0.001), loss='mse')

# Train the autoencoder
history = autoencoder.fit(X_train, X_train, epochs=100, batch_size=16, validat

# Define a secret key for encryption/decryption
secret_key = 42

# Encrypt and decrypt the test images
encrypted_images = np.array([encrypt_image(img, secret_key) for img in X_test])
decrypted_images = np.array([decrypt_image(img, secret_key) for img in encrypt

# Calculate metrics
avg_mse, avg_ssim, mse_values, ssim_values = calculate_metrics(X_test, decrypt
print(f"\nFinal Test Metrics - Mean Squared Error (MSE): {avg_mse:.6f}")
print(f"Final Test Metrics - Structural Similarity Index (SSIM): {avg_ssim:.6f}

# Display sample images (Original vs Encrypted vs Decrypted)
```

```
display_random_comparisons(X_test, encrypted_images, decrypted_images)

# Plot training loss
plot_training_loss(history)

# Plot MSE & SSIM distributions
plot_metrics_distribution(mse_values, ssim_values)
```

C:\Users\hitha sunil\anaconda3\Lib\site-packages\paramiko\transport.py:219: CryptographyDeprecationWarning: Blowfish has been deprecated
"class": algorithms.Blowfish,

Epoch 1/100

141/141  36s 221ms/step - loss: 0.0224 - val_loss: 0.0514

Epoch 2/100

141/141  31s 219ms/step - loss: 0.0133 - val_loss: 0.0154

Epoch 3/100

141/141  31s 221ms/step - loss: 0.0129 - val_loss: 0.0305

Epoch 4/100

141/141  31s 220ms/step - loss: 0.0125 - val_loss: 0.0144

Epoch 5/100

141/141  31s 219ms/step - loss: 0.0120 - val_loss: 0.0173

Epoch 6/100

141/141  31s 219ms/step - loss: 0.0115 - val_loss: 0.0196

Epoch 7/100

141/141  31s 218ms/step - loss: 0.0117 - val_loss: 0.0150

Epoch 8/100

141/141  31s 220ms/step - loss: 0.0111 - val_loss: 0.0227

```

In [2]: import os
import numpy as np
import random
from PIL import Image
from sklearn.model_selection import train_test_split
import tensorflow as tf
from tensorflow.keras.models import Model
from tensorflow.keras.layers import Input, Dense, Flatten, Reshape, BatchNorma
from tensorflow.keras.optimizers import Adam
from tensorflow.keras.callbacks import EarlyStopping
from skimage.metrics import structural_similarity as ssim
import matplotlib.pyplot as plt

# Function to Load images from a directory
def load_images_from_folder(folder, target_size=(128, 128)):
    images = []
    for filename in os.listdir(folder):
        img_path = os.path.join(folder, filename)
        try:
            img = Image.open(img_path).convert('L') # Convert to grayscale
            img = img.resize(target_size) # Resize to target size
            img = np.array(img).astype('float32') / 255.0 # Normalize to [0,
            images.append(img)
        except Exception as e:
            print(f"Error loading image {filename}: {e}")
    return np.array(images)

# Encrypt image using XOR encryption
def encrypt_image(image, key):
    np.random.seed(key) # Seed for reproducibility
    noise = np.random.rand(*image.shape) # Generate random noise
    return np.mod(image + noise, 1.0) # Add noise and take modulo to stay in

# Decrypt image using reverse XOR encryption
def decrypt_image(encrypted_image, key):
    np.random.seed(key) # Seed for reproducibility
    noise = np.random.rand(*encrypted_image.shape) # Generate same noise as e
    return np.mod(encrypted_image - noise, 1.0) # Subtract noise and take mod

# Build the autoencoder model with BatchNorm, LeakyReLU, and Dropout
def build_autoencoder(input_shape, latent_dim=128):
    # Encoder
    input_img = Input(shape=input_shape)
    x = Flatten()(input_img)
    x = Dense(512)(x)
    x = BatchNormalization()(x)
    x = LeakyReLU()(x)
    x = Dropout(0.3)(x) # Added Dropout
    x = Dense(latent_dim)(x)
    x = BatchNormalization()(x)
    x = LeakyReLU()(x)
    x = Dropout(0.3)(x) # Added Dropout
    encoder = Model(input_img, x, name="encoder")

    # Decoder

```

```

latent_input = Input(shape=(latent_dim,))
x = Dense(512)(latent_input)
x = BatchNormalization()(x)
x = LeakyReLU()(x)
x = Dropout(0.3)(x) # Added Dropout
x = Dense(np.prod(input_shape), activation='sigmoid')(x)
output_img = Reshape(input_shape)(x)
decoder = Model(latent_input, output_img, name="decoder")

# Autoencoder
autoencoder_input = Input(shape=input_shape)
encoded = encoder(autoencoder_input)
decoded = decoder(encoded)
autoencoder = Model(autoencoder_input, decoded, name="autoencoder")

return autoencoder, encoder, decoder

# Metrics calculation
def calculate_metrics(original_images, reconstructed_images):
    mse_values, ssim_values = [], []
    for orig, recon in zip(original_images, reconstructed_images):
        mse = np.mean((orig - recon) ** 2)
        ssim_value = ssim(orig, recon, data_range=orig.max() - orig.min())
        mse_values.append(mse)
        ssim_values.append(ssim_value)
    return np.mean(mse_values), np.mean(ssim_values), mse_values, ssim_values

# Display random image comparisons
def display_random_comparisons(original, encrypted, decrypted, num_samples=5):
    indices = random.sample(range(len(original)), num_samples) # Randomly sel
    plt.figure(figsize=(15, 5))

    for i, idx in enumerate(indices):
        # Original
        plt.subplot(3, num_samples, i + 1)
        plt.imshow(original[idx], cmap='gray', vmin=0, vmax=1)
        plt.title("Original")
        plt.axis('off')

        # Encrypted
        plt.subplot(3, num_samples, i + 1 + num_samples)
        plt.imshow(encrypted[idx], cmap='gray', vmin=0, vmax=1)
        plt.title("Encrypted")
        plt.axis('off')

        # Decrypted
        plt.subplot(3, num_samples, i + 1 + 2 * num_samples)
        plt.imshow(decrypted[idx], cmap='gray', vmin=0, vmax=1)
        plt.title("Decrypted")
        plt.axis('off')

    plt.tight_layout()
    plt.show()

# Plot training loss
def plot_training_loss(history):

```

```
plt.figure(figsize=(8, 5))
plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.title('Autoencoder Training Loss')
plt.legend()
plt.grid()
plt.show()

# Plot MSE & SSIM distributions
def plot_metrics_distribution(mse_values, ssim_values):
    plt.figure(figsize=(12, 5))

    # MSE distribution
    plt.subplot(1, 2, 1)
    plt.hist(mse_values, bins=20, color='skyblue', edgecolor='black')
    plt.xlabel('MSE')
    plt.ylabel('Frequency')
    plt.title('MSE Distribution')

    # SSIM distribution
    plt.subplot(1, 2, 2)
    plt.hist(ssim_values, bins=20, color='salmon', edgecolor='black')
    plt.xlabel('SSIM')
    plt.ylabel('Frequency')
    plt.title('SSIM Distribution')

    plt.tight_layout()
    plt.show()

# Load dataset
dataset_folder = "C:/Users/hitha sunil/Desktop/DOTA/DOTA/DOTA/DOTA_FULL" # Re
images = load_images_from_folder(dataset_folder)

# Split dataset into training and testing
X_train, X_test = train_test_split(images, test_size=0.2, random_state=42)

# Build and compile the autoencoder
input_shape = X_train[0].shape
autoencoder, encoder, decoder = build_autoencoder(input_shape)
autoencoder.compile(optimizer=Adam(learning_rate=0.001), loss='mse')

# Define early stopping
early_stopping = EarlyStopping(monitor='val_loss', patience=10, restore_best_w

# Train the autoencoder with early stopping
history = autoencoder.fit(
    X_train, X_train,
    epochs=100,
    batch_size=16,
    validation_data=(X_test, X_test),
    verbose=1,
    callbacks=[early_stopping] # Include early stopping
)
```

```
# Define a secret key for encryption/decryption
secret_key = 42

# Encrypt and decrypt the test images
encrypted_images = np.array([encrypt_image(img, secret_key) for img in X_test])
decrypted_images = np.array([decrypt_image(img, secret_key) for img in encrypted_images])

# Calculate metrics
avg_mse, avg_ssim, mse_values, ssim_values = calculate_metrics(X_test, decrypted_images)
print(f"\nFinal Test Metrics - Mean Squared Error (MSE): {avg_mse:.6f}")
print(f"Final Test Metrics - Structural Similarity Index (SSIM): {avg_ssim:.6f}")

# Display sample images (Original vs Encrypted vs Decrypted)
display_random_comparisons(X_test, encrypted_images, decrypted_images)

# Plot training Loss
plot_training_loss(history)

# Plot MSE & SSIM distributions
plot_metrics_distribution(mse_values, ssim_values)
```



```
Epoch 1/100
141/141  36s 222ms/step - loss: 0.0242 - val_loss: 0.0343
Epoch 2/100
141/141  31s 219ms/step - loss: 0.0150 - val_loss: 0.0142
Epoch 3/100
141/141  31s 220ms/step - loss: 0.0144 - val_loss: 0.0143
Epoch 4/100
141/141  31s 220ms/step - loss: 0.0141 - val_loss: 0.0147
Epoch 5/100
141/141  31s 219ms/step - loss: 0.0143 - val_loss: 0.0150
Epoch 6/100
141/141  31s 219ms/step - loss: 0.0139 - val_loss: 0.0140
Epoch 7/100
141/141  31s 220ms/step - loss: 0.0134 - val_loss: 0.0125
Epoch 8/100
141/141  31s 218ms/step - loss: 0.0132 - val_loss: 0.0171
Epoch 9/100
141/141  31s 219ms/step - loss: 0.0132 - val_loss: 0.0137
Epoch 10/100
141/141  31s 218ms/step - loss: 0.0132 - val_loss: 0.0126
Epoch 11/100
141/141  31s 218ms/step - loss: 0.0131 - val_loss: 0.0129
Epoch 12/100
141/141  31s 221ms/step - loss: 0.0130 - val_loss: 0.0117
Epoch 13/100
141/141  31s 220ms/step - loss: 0.0129 - val_loss: 0.0136
Epoch 14/100
141/141  31s 219ms/step - loss: 0.0125 - val_loss: 0.0118
Epoch 15/100
141/141  31s 220ms/step - loss: 0.0131 - val_loss: 0.0117
Epoch 16/100
141/141  31s 219ms/step - loss: 0.0130 - val_loss: 0.0151
Epoch 17/100
141/141  31s 219ms/step - loss: 0.0127 - val_loss: 0.0124
Epoch 18/100
141/141  31s 219ms/step - loss: 0.0125 - val_loss: 0.0113
Epoch 19/100
141/141  31s 221ms/step - loss: 0.0126 - val_loss: 0.0126
Epoch 20/100
141/141  31s 219ms/step - loss: 0.0127 - val_loss: 0.0142
Epoch 21/100
141/141  31s 221ms/step - loss: 0.0124 - val_loss: 0.0154
Epoch 22/100
141/141  31s 222ms/step - loss: 0.0125 - val_loss: 0.0117
Epoch 23/100
141/141  31s 221ms/step - loss: 0.0125 - val_loss: 0.0107
Epoch 24/100
141/141  31s 221ms/step - loss: 0.0124 - val_loss: 0.0143
Epoch 25/100
141/141  31s 218ms/step - loss: 0.0123 - val_loss: 0.0113
Epoch 26/100
141/141  31s 223ms/step - loss: 0.0125 - val_loss: 0.0122
Epoch 27/100
141/141  31s 219ms/step - loss: 0.0123 - val_loss: 0.0110
Epoch 28/100
141/141  31s 219ms/step - loss: 0.0122 - val_loss: 0.0110
```

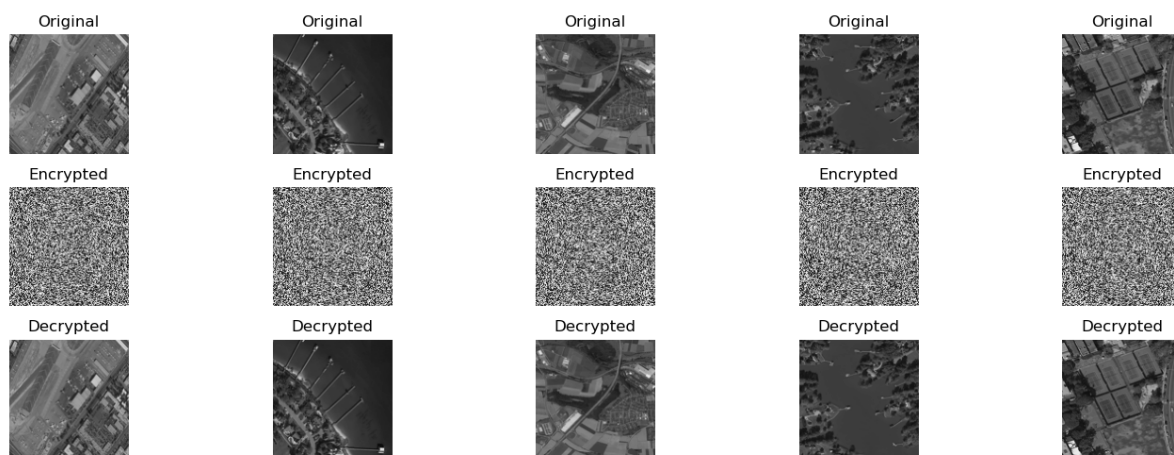
```
Epoch 29/100
141/141  31s 218ms/step - loss: 0.0125 - val_loss: 0.0120
Epoch 30/100
141/141  31s 219ms/step - loss: 0.0124 - val_loss: 0.0105
Epoch 31/100
141/141  31s 219ms/step - loss: 0.0118 - val_loss: 0.0116
Epoch 32/100
141/141  31s 220ms/step - loss: 0.0123 - val_loss: 0.0108
Epoch 33/100
141/141  31s 220ms/step - loss: 0.0120 - val_loss: 0.0109
Epoch 34/100
141/141  31s 222ms/step - loss: 0.0122 - val_loss: 0.0111
Epoch 35/100
141/141  41s 217ms/step - loss: 0.0118 - val_loss: 0.0110
Epoch 36/100
141/141  31s 219ms/step - loss: 0.0120 - val_loss: 0.0103
Epoch 37/100
141/141  31s 218ms/step - loss: 0.0123 - val_loss: 0.0102
Epoch 38/100
141/141  31s 221ms/step - loss: 0.0122 - val_loss: 0.0103
Epoch 39/100
141/141  32s 223ms/step - loss: 0.0119 - val_loss: 0.0104
Epoch 40/100
141/141  31s 218ms/step - loss: 0.0119 - val_loss: 0.0103
Epoch 41/100
141/141  31s 219ms/step - loss: 0.0115 - val_loss: 0.0106
Epoch 42/100
141/141  31s 219ms/step - loss: 0.0123 - val_loss: 0.0112
Epoch 43/100
141/141  31s 218ms/step - loss: 0.0122 - val_loss: 0.0111
Epoch 44/100
141/141  31s 218ms/step - loss: 0.0118 - val_loss: 0.0112
Epoch 45/100
141/141  31s 218ms/step - loss: 0.0119 - val_loss: 0.0102
Epoch 46/100
141/141  31s 218ms/step - loss: 0.0116 - val_loss: 0.0124
Epoch 47/100
141/141  31s 218ms/step - loss: 0.0116 - val_loss: 0.0112
Epoch 48/100
141/141  31s 218ms/step - loss: 0.0120 - val_loss: 0.0115
Epoch 49/100
141/141  31s 219ms/step - loss: 0.0117 - val_loss: 0.0110
Epoch 50/100
141/141  31s 218ms/step - loss: 0.0117 - val_loss: 0.0105
Epoch 51/100
141/141  31s 218ms/step - loss: 0.0121 - val_loss: 0.0102
Epoch 52/100
141/141  31s 218ms/step - loss: 0.0115 - val_loss: 0.0100
Epoch 53/100
141/141  31s 219ms/step - loss: 0.0118 - val_loss: 0.0109
Epoch 54/100
141/141  31s 218ms/step - loss: 0.0116 - val_loss: 0.0100
Epoch 55/100
141/141  31s 221ms/step - loss: 0.0117 - val_loss: 0.0104
Epoch 56/100
141/141  32s 223ms/step - loss: 0.0118 - val_loss: 0.0102
```

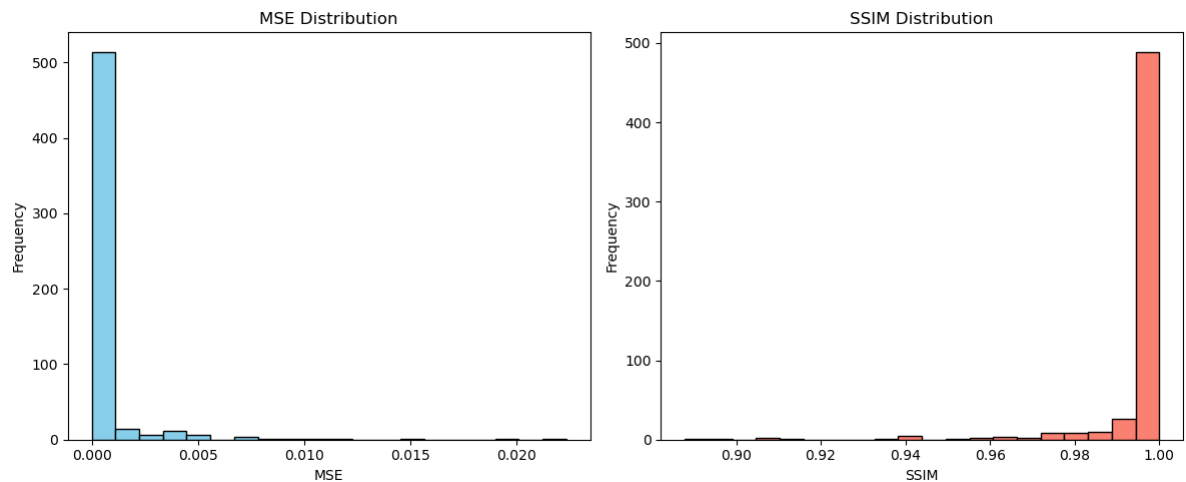
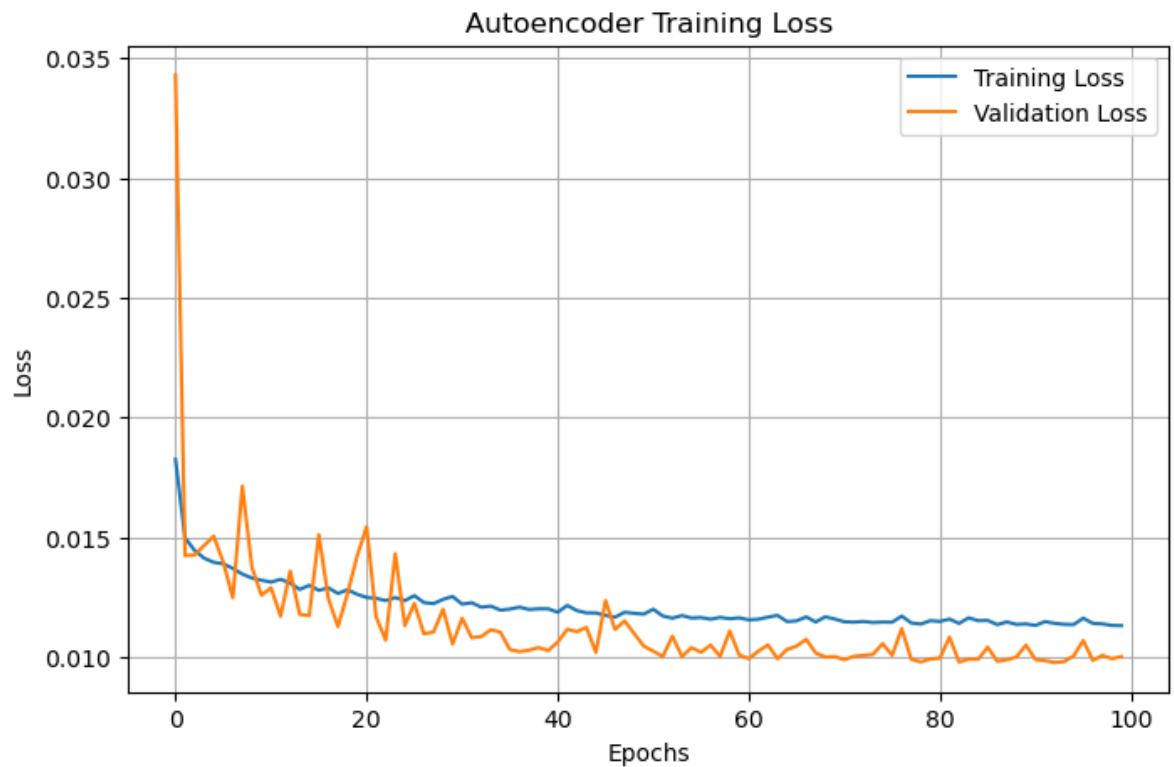
```
Epoch 57/100
141/141  30s 215ms/step - loss: 0.0115 - val_loss: 0.0105
Epoch 58/100
141/141  30s 215ms/step - loss: 0.0118 - val_loss: 0.0100
Epoch 59/100
141/141  30s 210ms/step - loss: 0.0115 - val_loss: 0.0111
Epoch 60/100
141/141  30s 210ms/step - loss: 0.0116 - val_loss: 0.0101
Epoch 61/100
141/141  30s 211ms/step - loss: 0.0115 - val_loss: 0.0099
Epoch 62/100
141/141  30s 209ms/step - loss: 0.0116 - val_loss: 0.0102
Epoch 63/100
141/141  30s 215ms/step - loss: 0.0115 - val_loss: 0.0105
Epoch 64/100
141/141  30s 213ms/step - loss: 0.0119 - val_loss: 0.0099
Epoch 65/100
141/141  30s 211ms/step - loss: 0.0116 - val_loss: 0.0103
Epoch 66/100
141/141  30s 210ms/step - loss: 0.0115 - val_loss: 0.0105
Epoch 67/100
141/141  29s 208ms/step - loss: 0.0118 - val_loss: 0.0107
Epoch 68/100
141/141  29s 209ms/step - loss: 0.0115 - val_loss: 0.0102
Epoch 69/100
141/141  29s 208ms/step - loss: 0.0114 - val_loss: 0.0100
Epoch 70/100
141/141  30s 209ms/step - loss: 0.0115 - val_loss: 0.0100
Epoch 71/100
141/141  30s 209ms/step - loss: 0.0111 - val_loss: 0.0099
Epoch 72/100
141/141  29s 208ms/step - loss: 0.0118 - val_loss: 0.0100
Epoch 73/100
141/141  29s 208ms/step - loss: 0.0115 - val_loss: 0.0101
Epoch 74/100
141/141  29s 208ms/step - loss: 0.0114 - val_loss: 0.0101
Epoch 75/100
141/141  29s 208ms/step - loss: 0.0112 - val_loss: 0.0106
Epoch 76/100
141/141  30s 209ms/step - loss: 0.0114 - val_loss: 0.0101
Epoch 77/100
141/141  29s 209ms/step - loss: 0.0116 - val_loss: 0.0112
Epoch 78/100
141/141  29s 209ms/step - loss: 0.0114 - val_loss: 0.0099
Epoch 79/100
141/141  29s 208ms/step - loss: 0.0115 - val_loss: 0.0098
Epoch 80/100
141/141  29s 209ms/step - loss: 0.0114 - val_loss: 0.0099
Epoch 81/100
141/141  30s 209ms/step - loss: 0.0111 - val_loss: 0.0100
Epoch 82/100
141/141  30s 209ms/step - loss: 0.0120 - val_loss: 0.0108
Epoch 83/100
141/141  30s 209ms/step - loss: 0.0109 - val_loss: 0.0098
Epoch 84/100
141/141  29s 209ms/step - loss: 0.0118 - val_loss: 0.0099
```

```
Epoch 85/100
141/141  29s 209ms/step - loss: 0.0114 - val_loss: 0.0099
Epoch 86/100
141/141  30s 210ms/step - loss: 0.0114 - val_loss: 0.0104
Epoch 87/100
141/141  30s 209ms/step - loss: 0.0113 - val_loss: 0.0098
Epoch 88/100
141/141  30s 210ms/step - loss: 0.0114 - val_loss: 0.0099
Epoch 89/100
141/141  30s 210ms/step - loss: 0.0111 - val_loss: 0.0100
Epoch 90/100
141/141  30s 210ms/step - loss: 0.0114 - val_loss: 0.0105
Epoch 91/100
141/141  30s 209ms/step - loss: 0.0112 - val_loss: 0.0099
Epoch 92/100
141/141  30s 209ms/step - loss: 0.0115 - val_loss: 0.0098
Epoch 93/100
141/141  30s 210ms/step - loss: 0.0116 - val_loss: 0.0098
Epoch 94/100
141/141  30s 210ms/step - loss: 0.0113 - val_loss: 0.0098
Epoch 95/100
141/141  29s 209ms/step - loss: 0.0112 - val_loss: 0.0101
Epoch 96/100
141/141  30s 209ms/step - loss: 0.0119 - val_loss: 0.0107
Epoch 97/100
141/141  30s 209ms/step - loss: 0.0111 - val_loss: 0.0099
Epoch 98/100
141/141  30s 209ms/step - loss: 0.0112 - val_loss: 0.0101
Epoch 99/100
141/141  29s 209ms/step - loss: 0.0111 - val_loss: 0.0099
Epoch 100/100
141/141  30s 209ms/step - loss: 0.0115 - val_loss: 0.0100
```

Final Test Metrics - Mean Squared Error (MSE): 0.000484

Final Test Metrics - Structural Similarity Index (SSIM): 0.996241





In [3]:

```
pip install --no-cache-dir h5py
```

Requirement already satisfied: h5py in c:\users\hitha sunil\anaconda3\lib\site-packages (3.12.1)
Requirement already satisfied: numpy>=1.19.3 in c:\users\hitha sunil\anaconda3\lib\site-packages (from h5py) (1.24.3)
Note: you may need to restart the kernel to use updated packages.

In []:

